

Question 7

Table of contents

Problem Statement	1
Analysis of Problem Statement	1
About the Algorithm	2
Algorithm	2
Implementation	4
Input file	5
Discussion	8

Problem Statement

Solve the following equation using *Gauss-Siedel* technique:

$$\begin{bmatrix} 1 & -1 & 2 \\ 2 & -2 & 3 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -8 \\ -20 \\ -2 \end{bmatrix}$$

Analysis of Problem Statement

- We make an assumption that an initial guess for the roots of the equation is provided as an input.
- We can have multiple kinds of stopping criteria, like:
 - A maximum number of iterations
 - A tolerance factor between two consecutive guesses
- We return the final guess as the solution to the equation, when it meets either stopping criteria.

About the Algorithm

- It tries to solve a system of linear equations of the form:

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1 : a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 = b_n$$

where x_i are the unknowns, a_{ij} are the coefficients of the equations, and b_i are the constants on the right-hand side of the equations.

- It takes an initial guess of x_i^0 , and iteratively updates the values of x_i using the formula:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right)$$

where, $x_i^{(k)}$ is the value of x_i at the k -th iteration.

- This can continue until the when criteria are met:
 - The maximum number of iterations is reached, and convergence doesn't occur.
 - The difference between the current and previous guesses is less than a specified tolerance.

Algorithm

- **Step 01. START**
- **Step 02.** Input: **A**, **b**, initial guess $\mathbf{x}^0$, maximum iterations $k_{\max}$ and tolerance factor ϵ
- **Step 03.** Initialize iteration_count $\mathbf{k} = 1$, current_guess $\mathbf{x}^{(k)} = \mathbf{x}^0$, next_guess $\mathbf{x}^{(k+1)} = \mathbf{0}$
- **Step 04.** Repeat until $k < k_{\max}$ or $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\| < \epsilon$:

– **Step 05.** Loop from $i = 1$ to n :

* **Step 06.** Update current_guess $\mathbf{x}^{(k)} = \mathbf{x}^{(k+1)}$

* **Step 07.** Update $x_i^{(k+1)}$ using the formula:

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij}x_j^{(k)} \right)$$

– **Step 08.** End loop {**Step 05**}

– **Step 09.** Increment iteration_count $k = k + 1$

- **Step 10.** End repeat {**Step 04**}
- **Step 11.** Output: Final guess $\mathbf{x}^{(k)}$
- **Step 12. STOP**

Implementation

```
%%file gaussssiedel.m
function [final_guess, converged, guess_history] = ...
    gaussssiedel(A, b, x0, epsilon, k_max) %#ok<FNDEF>

% @parameter A: coefficient matrix
% @parameter b: right-hand-side vector
% @parameter x0: initial guess vector
% @parameter k_max: maximum number of iterations
% @parameter epsilon: tolerance factor for convergence

% @return final_guess: final guess vector after convergence
% @return converged: shows if convergence was achieved
% @return guess_history: history of guesses

if nargin < 5; k_max = 100; end
if nargin < 4; epsilon = 1e-6; end

k = 1; current_guess = x0; new_guess = inf(size(x0));
guess_history = zeros(k_max, size(x0, 1));
guess_history(1, :) = x0;

convergence_criteria_met = @(current_guess, new_guess) ...
    norm(new_guess - current_guess) < epsilon;
stopping_criteria_met = @(iter, current_guess, new_guess) ...
    iter >= k_max || convergence_criteria_met(current_guess, new_guess);

while ~stopping_criteria_met(k, current_guess, new_guess)
    if k > 1; current_guess = new_guess; end

    for i = 1:size(A, 1)
        terms_before = A(i, 1:i-1) * new_guess(1:i-1);
        terms_after = A(i, i+1:end) * current_guess(i+1:end);
        new_guess(i) = (b(i) - terms_before - terms_after) / A(i, i);
        fprintf("Iteration %d - x%d: %.6f\n", k, i, new_guess(i));
    end % end of loop updating new guess values for one iteration

    k = k + 1;
    guess_history(k, :) = new_guess;
end % end of loop of iterations predicting guess vectors

final_guess = new_guess;
converged = convergence_criteria_met(current_guess, new_guess);
guess_history = guess_history(1:k, :);
end
```

File gaussssiedel.m created successfully.

Input file

We add a file with the input in a neatly organized manner:

A:

```
1  -1  2
2  -2  3
1   1  1
```

B:

```
-8
-20
-2
```

Initial guess:

```
0
0
0
```

```

% --- INPUT ---
fprintf("--- INPUT ---\n");
INPUT_FILE = "input.txt";

file = fopen(INPUT_FILE, "r");
while ~feof(file)
    line = fgetl(file);

    if startsWith(line, "n")
        n = sscanf(line, "n = %f");
    elseif startsWith(line, "A")
        A = fscanf(file, "%f", [n, n]);
    elseif startsWith(line, "B")
        b = fscanf(file, "%f", [n, 1]);
    elseif startsWith(line, "Initial guess")
        x0 = fscanf(file, "%f", [n, 1]);
    end
end
fclose(file);

display_input = table(A, b, x0, ...
    VariableNames={'A', 'b', 'Initial_guess'});
disp(display_input);

% --- OUTPUT ---
fprintf("\n--- OUTPUT ---\n");
[final_guess, converged, guess_history] = gaussssiedel(A, b, x0, 1e-6, 10);

fprintf("Final guess: \n");
format short g
disp(final_guess);

fprintf("Converged: %s\n", string(converged));

% print pretty table of guess history, with truncation for too many iterations
row_indices = "Iteration " + string(1:size(guess_history, 1));
fprintf("\nGuess history:\n");
guess_history_table = array2table(guess_history, ...
    VariableNames="x" + (1:n), RowNames=row_indices);
disp(guess_history_table);

```

--- INPUT ---

	A		b	Initial_guess
	-----		-	-----
10	-1	2	4	0
1	10	-1	3	0
2	3	20	7	0

--- OUTPUT ---

Iteration 1 - x1: 0.400000
Iteration 1 - x2: 0.260000
Iteration 1 - x3: 0.271000
Iteration 2 - x1: 0.371800
Iteration 2 - x2: 0.289920
Iteration 2 - x3: 0.269332
Iteration 3 - x1: 0.375126
Iteration 3 - x2: 0.289421
Iteration 3 - x3: 0.269074
Iteration 4 - x1: 0.375127
Iteration 4 - x2: 0.289395
Iteration 4 - x3: 0.269078
Iteration 5 - x1: 0.375124
Iteration 5 - x2: 0.289395
Iteration 5 - x3: 0.269078
Iteration 6 - x1: 0.375124
Iteration 6 - x2: 0.289395
Iteration 6 - x3: 0.269078
Final guess:
0.37512
0.2894
0.26908

Converged: true

Guess history:

	x1	x2	x3
	-----	-----	-----
Iteration 1	0	0	0
Iteration 2	0.4	0.26	0.271
Iteration 3	0.3718	0.28992	0.26933
Iteration 4	0.37513	0.28942	0.26907

Iteration 5	0.37513	0.28939	0.26908
Iteration 6	0.37512	0.2894	0.26908
Iteration 7	0.37512	0.2894	0.26908

Discussion

Gauss-Seidel is guaranteed to converge only if the coefficient matrix $\mathbf{A}$ is: - $\mathbf{A}$ is symmetric positive-definite, or - $\mathbf{A}$ is strictly diagonally dominant.

Diagonal dominance means that for every row of the matrix i :

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}|$$

For our matrix, every row fails this test. Thus the error grows instead of shrinking — looking at the guess history, where values increase each iteration instead of settling near the true solution.

One place to improve further — the current algorithm stops based either on `k_max` or tolerance but does not have a way to see if it diverges early enough. Including a condition that would check if $\|x^{(k+1)} - x^{(k)}\|$ is increasing (or a diagonal dominance test initially) would allow the algorithm to stop early.